

VegZo

Complete Deployment & Production Guide

AWS · DynamoDB · PWA · Google Play · App Store

Version	v1.0
Date	2024
Cloud	AWS us-east-1 (America)
Apps	Customer · Delivery Partner · Admin
Scale	1000+ concurrent users

Confidential — Internal Use Only

1. System Architecture

VegZo uses AWS serverless infrastructure — no servers to manage, auto-scaling, and pay-per-use pricing. All three app pages (Customer, Delivery, Admin) are served from S3 via CloudFront. API calls go through API Gateway → Lambda → DynamoDB.

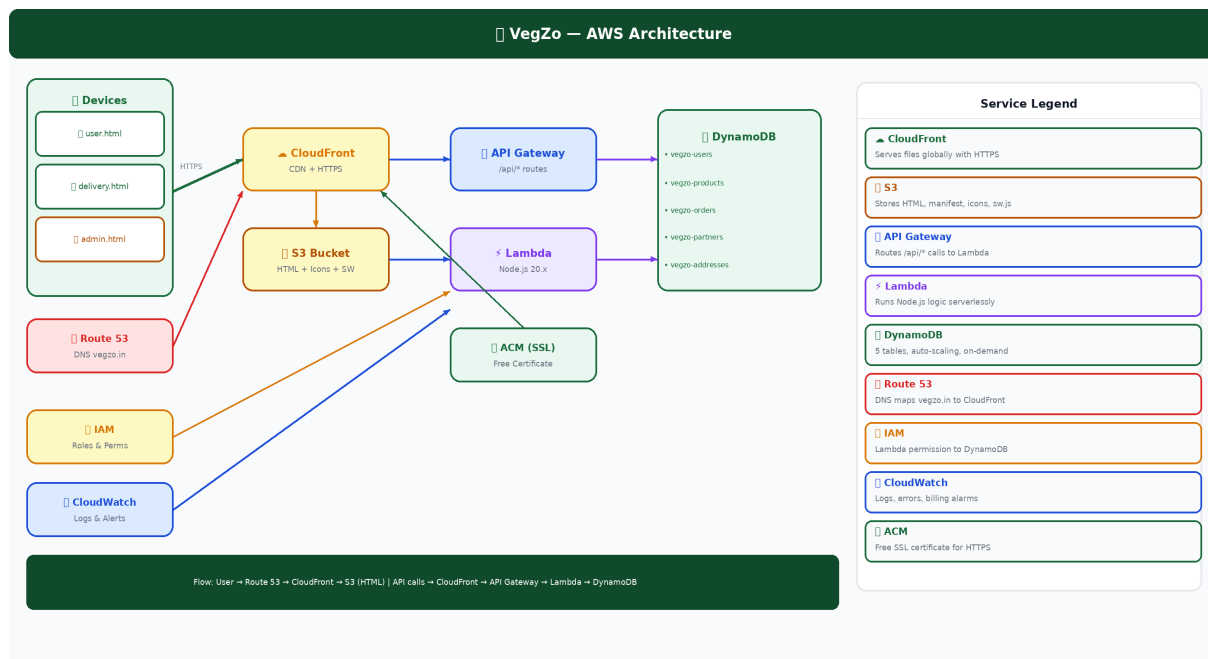


Figure 1: VegZo AWS Architecture — Full cloud service map

1.1 AWS Services Overview

Service	Purpose	Details
S3	Static hosting	Stores user.html, delivery.html, admin.html, manifest, icons
CloudFront	CDN + HTTPS	Global delivery, SSL termination, edge caching
API Gateway	REST API	Routes /api/* requests from frontend to Lambda
Lambda	Business logic	Node.js 20.x — all CRUD operations, runs on demand
DynamoDB	Database	5 tables: users, products, orders, partners, addresses
Route 53	DNS	Maps vegzo.in to your CloudFront distribution
ACM	SSL Certificate	Free HTTPS certificate — required for PWA + stores
IAM	Security	Lambda role with DynamoDB read/write permissions
CloudWatch	Monitoring	Logs, errors, latency metrics, billing alarms

2. DynamoDB Setup

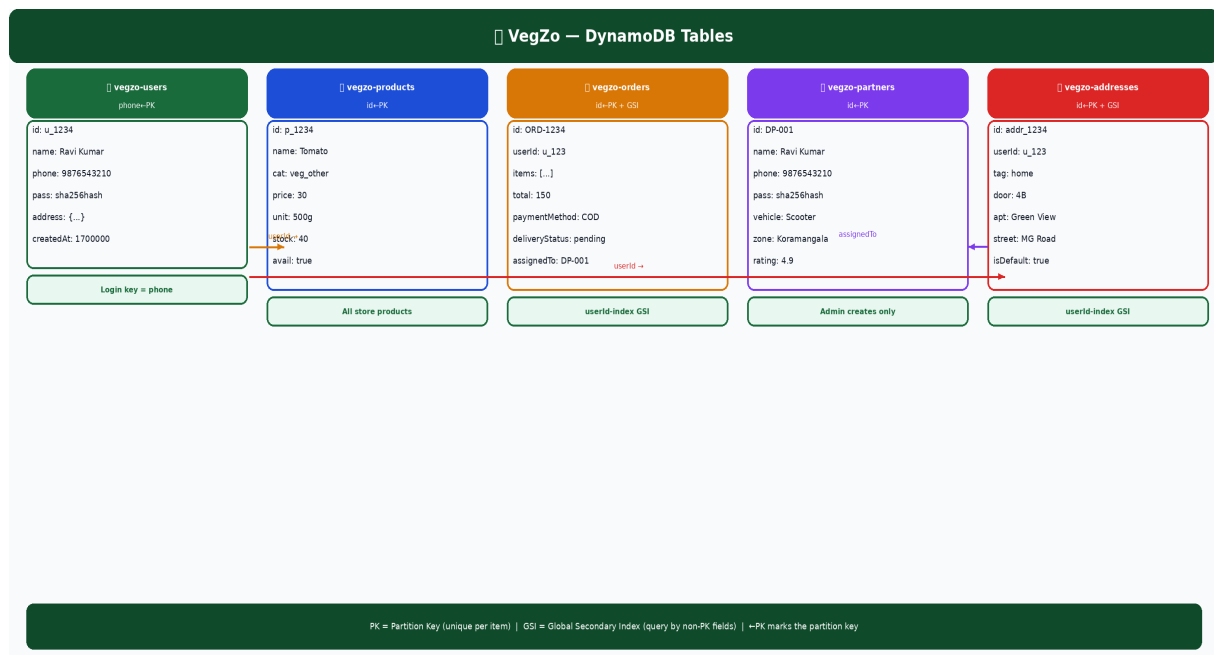


Figure 2: DynamoDB — 5 tables, partition keys, GSIs and relationships

2.1 Create the 5 Tables

Go to: AWS Console → DynamoDB → Tables → Create table. Use On-Demand capacity for all tables.

Table Name	Partition Key	GSI Index	Purpose
vegzo-users	phone (String)	none	Customer accounts — login by phone
vegzo-products	id (String)	none	Products: vegetables, fruits, drinks, grains
vegzo-orders	id (String)	userId-index	Orders — queryable by userId
vegzo-partners	id (String)	none	Delivery partners — created by admin only
vegzo-addresses	id (String)	userId-index	Address book — queryable by userId

2.2 Add GSI Indexes (Important!)

- Open **vegzo-orders** → Indexes tab → Create index → Partition key: userId (String) → Name: userId-index
- Open **vegzo-addresses** → same steps → userId-index

3. Lambda + API Gateway

3.1 Create Lambda Function

■ AWS Console → Lambda → Create function → Author from scratch

1. Function name: **vegzo-api-handler**
2. Runtime: **Node.js 20.x** | Architecture: **arm64** (40% cheaper)
3. Click Create function

3.2 Upload Code

1. On your computer: create folder **vegzo-lambda/** → copy **lambda.js** inside
2. In Terminal: **npm init -y** then **npm install @aws-sdk/client-dynamodb @aws-sdk/lib-dynamodb**
3. Zip the folder: **zip -r vegzo-lambda.zip lambda.js node_modules/**
4. Lambda console → Upload from .zip → Handler: **lambda.handler**

3.3 Environment Variables

Key	Value	Purpose
SALT	your_secret_random_key_here	Password hashing salt — keep secret!
AWS_REGION	us-east-1	North Virginia region for DynamoDB

3.4 IAM + Timeout

1. Configuration → Permissions → click Role name → Attach: **AmazonDynamoDBFullAccess**
2. Configuration → General → Timeout: **30 seconds** | Memory: **256 MB**

3.5 API Gateway

■ AWS Console → API Gateway → Create API → REST API → Build

1. API name: **vegzo-api** | Endpoint type: Regional
2. Create resource: **/api** (enable CORS) → sub-resources: auth, products, orders, partners, addresses
3. For each resource create methods: POST, GET, PUT, DELETE → Integration: **vegzo-api-handler**
4. Actions → Enable CORS on every resource
5. Actions → Deploy API → Stage: **prod** → Click Deploy → **Copy the Invoke URL!**

■ CRITICAL: Open **api-client.js** → replace **YOUR_API_ID** in **API_BASE** with your actual API Gateway ID.

4. S3 + CloudFront + Domain

4.1 Create S3 Bucket

1. S3 → Create bucket → Name: **vegzo-app-2024** | Region: ap-south-1
2. Uncheck "Block all public access" → acknowledge → Create
3. Properties → Static website hosting → Enable → Index: **user.html**
4. Permissions → Bucket policy → paste the public-read policy
5. Upload: user.html, delivery.html, admin.html, manifest.json, sw.js, icon-192.png, icon-512.png, api-client.js

4.2 CloudFront

1. CloudFront → Create distribution → Origin: your S3 website endpoint (not ARN)
2. Viewer protocol: **Redirect HTTP to HTTPS** | Default root: **user.html**
3. Wait 10 min → Status: Deployed → copy **d1abc.cloudfront.net**

4.3 Domain + SSL

1. Route 53 → Register domain → vegzo.in (~\$700/yr) or vegzo.com
2. Switch region to **us-east-1** → Certificate Manager → Request certificate
3. Domains: vegzo.in + *.vegzo.in → DNS validation → Create records in Route 53
4. CloudFront → Edit → Alternate domain: vegzo.in → attach SSL cert → Save
5. Route 53 → Create A record → Alias to CloudFront distribution

■ Your app is now live at <https://vegzo.in> with full HTTPS. Admin page: <https://vegzo.in/admin.html>

5. PWA — Make It Installable

PWA (Progressive Web App) makes your web app installable on Android and iPhone like a native app.

5.1 Add to user.html and delivery.html (inside <head>)

```
<link rel="manifest" href="/manifest.json"/>
<meta name="theme-color" content="#1a6b3c"/>
<meta name="mobile-web-app-capable" content="yes"/>
<meta name="apple-mobile-web-app-capable" content="yes"/>
<link rel="apple-touch-icon" href="/icon-192.png"/>
```

5.2 Register Service Worker (before </body>)

```
<script>
if ('serviceWorker' in navigator) {
window.addEventListener('load', () => {
navigator.serviceWorker.register('/sw.js');
});
}
</script>
```

5.3 Required Icons

File	Size	Required For
icon-192.png	192×192	Android home screen (REQUIRED)
icon-512.png	512×512	Play Store + splash screen (REQUIRED)
icon-delivery-192.png	192×192	Delivery app icon
icon-delivery-512.png	512×512	Delivery Play Store listing

■ Use [squoosh.app](#) (free) to resize your VegZo logo PNG to each required size.

6. User & Order Flow

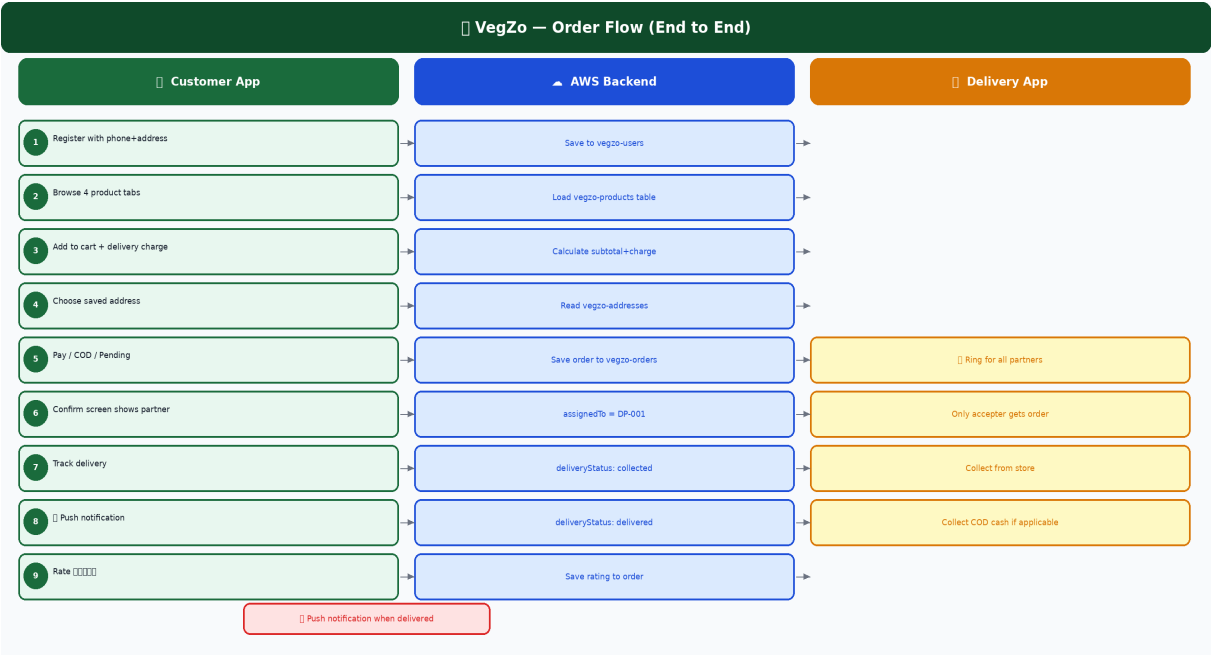


Figure 3: End-to-end order flow — Customer App, AWS Backend, Delivery Partner App

6.1 Payment & Order Status

Payment Method	Order Status	Delivery Partner Sees	User Sees
UPI / Card (paid)	placed	Normal order	Payment Done! ■
Cash on Delivery	placed	COD badge + collect cash	Order Placed — keep cash ready
Payment failed	payment_pending	Not visible yet	Pay Now or COD option
Save Order (pay later)	payment_pending	Not visible yet	Pay Now or COD in Order History

6.2 Ring Notification Flow

- New order arrives → ALL online delivery partners get a ring alert
- First partner to tap **Accept** claims the order
- Ringtone plays using Web Audio API — no file download needed
- Other partners see the order greyed out: "Claimed by [name]"
- COD orders show a ■ yellow badge so partner knows to collect cash

7. Scalability — 1,000+ Concurrent Users

The current localStorage-based architecture works great for development and small teams. For 1,000+ simultaneous users in production, here is the upgraded architecture:

7.1 Why AWS Serverless Scales Automatically

Component	Scales How	Max Capacity
CloudFront	Edge CDN — serves HTML from 400+ locations worldwide	Unlimited
API Gateway	Managed — auto-scales with traffic spikes	10,000 RPS default
Lambda	Auto-scales: 1 request = 1 instance, up to 1000 parallel	1000 concurrent
DynamoDB	On-Demand mode: scales instantly, no provisioning	Unlimited RCU/WCU
S3	Object storage — serves files globally via CloudFront	Unlimited

7.2 Optimizations for 1000 Users

Optimization	What to Do	Impact
DynamoDB Caching	Add ElastiCache (Redis) in front of DynamoDB for product lookups	Product load 10x faster
Lambda Warmup	Set Lambda provisioned concurrency = 10 to avoid cold starts	Faster response time
CloudFront Cache	Set cache-control headers on HTML files (max-age=300)	Reduces origin hits 80%
Image CDN	Store product images in S3 + serve via CloudFront with ImageKeyVersion	Page load 3x faster
DynamoDB Streams	Use Streams to trigger order notifications in real-time	Instant order updates
API Rate Limiting	Set API Gateway throttle: 100 RPS per IP to prevent abuse	Prevents overload
Connection Pooling	Lambda → RDS Proxy if switching to SQL later	Handles more queries
SQS Queue	Use SQS for order processing to handle traffic bursts	Zero order drops

7.3 Load Test Targets

Metric	Target	How to Test
Page Load Time	< 2 seconds	Google PageSpeed Insights
API Response Time	< 200ms p95	AWS CloudWatch metrics
Concurrent Users	1,000+	Artillery.io load testing
Order Processing	< 500ms end-to-end	Custom Lambda metrics
Uptime	99.9% SLA	AWS Status Dashboard

■ AWS Free Tier supports up to 1M Lambda calls/month. For 1000 users placing 10 orders/day = 300K calls/month — well within free tier!

8. Payment & Order Logic

8.1 What Happens When Payment Fails

Scenario	What User Sees	What Delivery Partner Sees
UPI app installed	Redirected to app → verify → success	Order appears after payment success
UPI app not installed	Install app prompt + "Use another method"	Not visible
Payment declined (25%)	"Payment Failed" screen with options	Not visible
User clicks Pay Later	Order saved — "Payment Pending" shown in Order History	Not visible
User converts to COD	Order confirmed with COD	COD badge + collect cash prompt

8.2 Cash on Delivery Flow — Delivery Partner

- COD orders show a yellow ■ **COD** badge in the order card header
- When status is "Collected" — a gold banner shows: "**Collect ■X Cash**" before handing over
- Button reads "**■ Cash Collected — Mark Delivered**" (requires confirmation)
- Order is marked delivered only after partner confirms cash collection

8.3 Ring Notification

- New orders trigger a Web Audio API ringtone — no MP3 file needed
- Musical pattern (C-E-G scale) plays every 2.2 seconds until accepted/declined
- **Only partner who accepts** gets the order — prevents double-claiming
- After accept, ringtone stops and order is claimed instantly